

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 11 — OBSERVABILITY & DEPLOYMENT

Prerequisites: Phases 1–10 done. You have chains and/or agents you'd like to debug and serve.

Targets: LangSmith tracing · reading a trace · serving a chain via FastAPI · LangGraph Platform · production checklist

Estimated time for this phase: 75–110 minutes

Where this phase sits

Your apps work on your machine. Two things stand between that and production: **seeing** what they do (observability) and **serving** them to others (deployment). This phase covers both — LangSmith tracing, and wrapping a chain in a real web API.

Ph 0–4 ✓ **Setup** → **Fundamentals** → **Models** → **Prompts** → **Chains**

Ph 5–8 ✓ **Memory** · **Documents** · **Embeddings** · **RAG**

Ph 9 ✓ **Tools & Agents**

Ph 10 ✓ **LangGraph Basics** — nodes, edges, branches, loops.

Ph 11 **Observability & Deployment** ← you are here
LangSmith tracing, reading traces, serving a chain via FastAPI, deployment options.

Ph 12 **Capstone Project** — everything, end to end.

Phase 11 · Observability & Deployment

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Turn on LangSmith tracing with two environment variables and read a trace to debug any chain or agent; wrap a chain in a FastAPI endpoint and call it over HTTP; know the current deployment options (FastAPI vs LangGraph Platform, and why LangServe is now legacy); and run through a production-readiness checklist.

In this phase

1. 11.1 — The two production concerns
2. 11.2 — LangSmith: seeing inside your app
3. 11.3 — Turning on tracing (no code changes)
4. 11.4 — Reading a trace & tracing your own functions
5. 11.5 — Deployment: from script to service
6. 11.6 — Serving a chain with FastAPI
7. 11.7 — Production-readiness checklist
8. 11.8 — Common errors & troubleshooting
9. 11.9 — Mini practice exercise
10. Terms introduced this phase

11.1 — The two production concerns

A working script is not a product. To run for real you need to answer two questions that never came up while developing:

- **Observability:** "When this misbehaves in production, how do I see *what* happened — which step was slow, what the model actually received, where it errored?" Print statements don't scale.
- **Deployment:** "How does someone *other than me* use this — a web app, a teammate, a mobile client?" A Python file on your laptop can't be called over the internet.

LangSmith answers the first; a web framework (FastAPI) or managed platform answers the second.

11.2 — LangSmith: seeing inside your app

TERM: LANGSMITH & TRACING

LangSmith is LangChain's hosted observability platform. **Tracing** means recording every step of a run — each model call, prompt, tool, and retrieval — with its inputs, outputs, latency, token usage, and any errors, laid out as a nested timeline. It's the upgrade from the `set_debug` / `peek` tricks of Phase 4: instead of cluttering your console, you get a searchable visual record of every run.

WHY IT'S WORTH SETTING UP EARLY

RAG and agents are multi-step and partly non-deterministic — exactly the things that are painful to debug blind. With a trace you can see that the retriever returned the wrong chunks (Phase 8's "retrieval-first" rule), or that an agent looped five times, or which prompt ballooned your token bill. It also tracks cost and latency over time. For anything beyond a toy, it pays for itself fast.

11.3 — Turning on tracing (no code changes)

The remarkable part: if you're using LangChain/LangGraph, tracing needs **zero code changes**. You set environment variables and LangChain auto-reports every run.

1. Sign up free at smith.langchain.com and create an API key (Settings → API Keys).
2. Install the SDK: `pip install -U langsmith`
3. Add these to your `.env` file (alongside your model keys from Phase 0):

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=ls_your_key_here
LANGSMITH_PROJECT=langchain-course      # optional: groups your runs (default: "default")
```

That's it. Now run *any* chain or agent from earlier phases as usual:

```
from dotenv import load_dotenv
load_dotenv()                    # loads the LANGSMITH_* vars too

# ... your existing RAG chain or agent ...
result = rag_chain.invoke("How do I get a refund?")
```

Open smith.langchain.com, pick your project, and the run appears with every step expanded. No tracing code in your app at all.

VARIABLE NAMES

The current names are `LANGSMITH_TRACING` / `LANGSMITH_API_KEY` / `LANGSMITH_PROJECT`. You'll still see the older `LANGCHAIN_TRACING_V2` / `LANGCHAIN_API_KEY` / `LANGCHAIN_PROJECT` in tutorials — those still work, but prefer the `LANGSMITH_*` names. There's also a `LANGSMITH_ENDPOINT` (defaults to the US cloud; an EU endpoint exists).

11.4 — Reading a trace & tracing your own functions

A trace is a tree. The top is the whole run; expand it to see each step nested inside, in order. For each step you get:

In a trace you can see...	Useful for
The exact input & output of every step	Confirming the prompt the model really got, or the chunks the retriever returned.

Latency per step	Finding the slow part of a pipeline.
Token counts & cost	Spotting an oversized prompt or runaway agent.
Errors & stack traces	Seeing exactly which step failed and why.
The full message sequence for agents	Watching the reason-act loop turn by turn (Phase 9).

To include a *non-LangChain* function in the trace (e.g. your own pre-processing), decorate it with `@traceable`:

```
from langsmith import traceable

@traceable # this function now shows up as a step in the trace
def clean_text(text: str) -> str:
    return text.strip().lower()
```

11.5 — Deployment: from script to service

TERM: DEPLOYMENT

Deployment means putting your app somewhere it can run continuously and be called by others — typically by exposing it as a **web API** (an HTTP endpoint a client sends requests to). Your `chain.invoke()` becomes "POST a question to `/ask`, get an answer back."

Three current options, simplest to most managed:

Option	What it is	Best when
FastAPI (we'll use this)	A standard Python web framework. You write a small endpoint that calls your chain.	You want full control and to host anywhere. The general-purpose choice.
LangGraph Platform	LangChain's managed service for deploying LangGraph apps (agents/graphs) with persistence, scaling, and a built-in API.	You built a LangGraph agent/graph and want managed hosting with less ops work.
LangServe	Older library that auto-wraps a Runnable as an API.	Legacy only — see warning below.

⚠️ LANGSERVE IS NOW LEGACY

LangServe is in maintenance mode — bug fixes only, no new features — and LangChain points users to **LangGraph Platform** for deploying LangGraph apps, or plain **FastAPI** for custom serving. Don't start a new project on LangServe. We teach FastAPI because it's vendor-neutral, ubiquitous, and works for any chain or agent.

11.6 — Serving a chain with FastAPI

Install the tools: `pip install fastapi uvicorn` (`uvicorn` is the server that runs the app). Then create `server.py`:

```
from fastapi import FastAPI                                # 1
from pydantic import BaseModel
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

chain = (                                                    # 2 any chain from earlier
    ChatPromptTemplate.from_template("Explain {topic} in one sentence.")
    | ChatGoogleGenerativeAI(model="gemini-2.5-flash")
    | StrOutputParser()
)

app = FastAPI()                                              # 3 the web app

class Query(BaseModel):                                     # 4 the expected request body
    topic: str

@app.post("/ask")                                           # 5 an endpoint at POST /ask
def ask(q: Query):
    answer = chain.invoke({"topic": q.topic})               # 6 run the chain on the request
    return {"answer": answer}                               # 7 JSON response
```

Line	Explanation
2	Build the chain once, at startup — not per request. (Swap in your RAG chain or wrap an <code>agent</code> just as easily; for an agent call <code>agent.invoke({"messages": [...]})</code> .)
3	<code>FastAPI()</code> creates the web application object.
4	A Pydantic model (Phase 3!) defines the request shape. FastAPI validates incoming JSON against it automatically.
5–7	<code>@app.post("/ask")</code> registers a handler for HTTP POST to <code>/ask</code> . It runs the chain on the request's <code>topic</code> and returns JSON.

Run the server:

```
uvicorn server:app --reload
```

(`server` = the filename, `app` = the FastAPI object; `--reload` restarts on code changes.) Then call it:

```
curl -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"topic": "gravity"}'
```

Expected response:

```
{"answer": "Gravity is the invisible force that pulls objects with mass toward each other."}
```

FREE INTERACTIVE DOCS

FastAPI auto-generates a test UI. Visit <http://localhost:8000/docs> in a browser to see your `/ask` endpoint and try it with a form — no curl needed. Great for sharing with teammates.

STREAMING RESPONSES

For a typing-effect API, return a streaming response that yields from `chain.stream(...)` using FastAPI's `StreamingResponse`. The mechanics are beyond this intro, but know it's a small addition — the chain already supports `.stream` (Phase 2), so the endpoint just forwards the chunks.

11.7 — Production-readiness checklist

Before exposing an app to real users, walk this list:

Concern	What to do
Secrets	Never hard-code keys. Use environment variables / a secrets manager — the <code>.env</code> habit from Phase 0, plus <code>.gitignore</code> .
Observability	Enable LangSmith in the deployed environment (set the env vars there too) so you can debug production runs.
Persistence	For memory/agents, swap <code>InMemorySaver</code> for a database checkpointer (SQLite/Postgres, Phase 5) so state survives restarts and scales across servers.
Errors & timeouts	Wrap calls in error handling; set model <code>timeout</code> / <code>max_retries</code> (Phase 2). Return clean error responses, not stack traces.
Rate & cost limits	Cap request rates; watch token usage in LangSmith; consider a cheaper/local model for high volume.
Concurrency	Use <code>async</code> endpoints and the <code>a*</code> methods (<code>ainvoke</code> , Phase 2) under heavy load; run multiple uvicorn workers.
Input validation	Pydantic request models (as above) reject malformed input before it reaches the model.

11.8 — Common errors & troubleshooting

Symptom	Likely cause & fix
No traces appear in LangSmith	<code>LANGSMITH_TRACING</code> not <code>true</code> , key missing/wrong, or <code>load_dotenv()</code> didn't run before the chain. Check the env vars are loaded in <i>this</i> process.
Traces go to the wrong/"default" project	Set <code>LANGSMITH_PROJECT</code> . It groups runs; unset means everything lands in <code>default</code> .
<code>uvicorn: command not found</code>	<code>pip install uvicorn</code> (and <code>fastapi</code>); ensure your venv is active.
<code>422 Unprocessable Entity</code> from the API	Request JSON doesn't match the Pydantic model (wrong/missing field). Match the keys; check <code>/docs</code> for the expected shape.
Server works locally but not from another machine	Bind to all interfaces: <code>uvicorn server:app --host 0.0.0.0 --port 8000</code> , and open the firewall/port.
Chain rebuilt on every request (slow)	Build the chain/model/store at module load (once), not inside the endpoint function.
Followed a LangServe tutorial and hit dead ends	LangServe is legacy (§11.5). Use FastAPI, or LangGraph Platform for LangGraph apps.

11.9 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 12)

Task: Take your Phase 8 RAG chain (or Phase 9 agent) and (1) enable LangSmith tracing via env vars, run a few queries, and explore the traces — find the retrieval step and confirm which chunks came back; then (2) wrap the same chain in a FastAPI `/ask` endpoint, run it with uvicorn, and call it from `/docs` and with curl. Bonus: add a second endpoint `/health` that returns `{"status": "ok"}` for uptime checks.

Why: this is the difference between "a script that works" and "a service you can ship." Seeing your RAG pipeline in a trace will also sharpen the retrieval-first debugging instinct from Phase 8 — and you'll reuse this exact server skeleton for the Phase 12 capstone.

Hints (not the full answer):

- Tracing first needs nothing but the env vars + `load_dotenv()` — verify a trace appears before touching FastAPI.
- For a RAG endpoint, the request model is just `class Query(BaseModel): question: str`; call `rag_chain.invoke(q.question)`.
- Build the store/retriever/chain once at the top of `server.py`, outside the endpoint function.
- In a trace, click into the retriever step to see the returned documents — compare against what the model answered.

Terms introduced this phase

Term	One-line meaning
Observability	Being able to see what your app did internally — steps, inputs/outputs, timing, errors.
LangSmith	LangChain's hosted platform for tracing, debugging, and monitoring LLM apps.
Trace	A recorded, nested timeline of one run: every step with its data, latency, tokens.
@traceable	Decorator that adds your own function to a LangSmith trace.
LANGSMITH_TRACING / _API_KEY / _PROJECT	Env vars that switch on tracing, authenticate, and group runs.
Deployment	Running your app where others can call it, usually as a web API.
API endpoint	A URL (e.g. <code>POST /ask</code>) a client sends requests to.
FastAPI / uvicorn	A Python web framework / the server program that runs it.
LangGraph Platform	Managed hosting for LangGraph agents/graphs with built-in persistence & scaling.
LangServe (legacy)	Older auto-API library, now maintenance-only; prefer FastAPI / LangGraph Platform.

PHASE 11 COMPLETE

You can switch on LangSmith tracing with two env vars and debug any run visually, and you can serve a chain or agent as a real HTTP API with FastAPI — plus you know the current deployment landscape and a checklist for going live. **Next — Phase 12: Capstone Project**, where you combine everything — RAG, memory, tools, an agent, tracing, and a served API — into one complete document-Q&A assistant with swappable Gemini/Ollama backends. The course also wraps with a full glossary, a "what to learn next" guide, and the documentation references.